4-bit array multiplier from RTL to GDSII

Krish Pandya (2023102026) and Vignesh Vembar (2023102019)

Digital VLSI Design, SkyWater SKY130, `sky130_fd_sc_hd`

1 RTL design and verification

`rtl/full_adder.v` computes `sum = a ^ b ^ cin` and `cout = (a & b) | (a & cin) | (b & cin)` with two assign statements. `rtl/mult_array.v` registers `a` and `b` into `a_reg` and `b_reg`, forms the 16 partial-product bits `pp[i][j] = a_reg[j] & b_reg[i]`, adds them in three rows of four `full_adder` instances (12 in total) with the connections of Section 2 of the brief, and registers the 8-bit result into `p`. All 16 flip-flops reset to 0 when `rst_n` is low. Figure 1 shows the array.

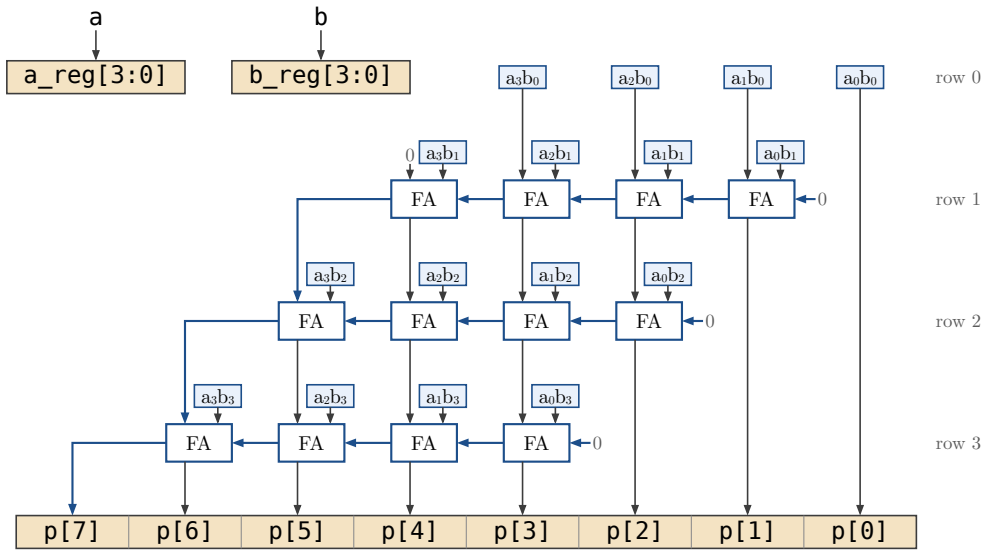


Figure 1: Block diagram. Each column is one bit weight. A gate $a_j b_i$ is the AND of `a_reg[j]` and `b_reg[i]`. Each row of full adders (FA) adds one partial product, so it sits one column left of the row above. Sums (grey) drop straight down and carries (blue) run right to left.

1.1 Testbench

`tb/tb_mult.v` runs a 10 ns clock and holds `rst_n` low from 0 to 20 ns, the first 2 cycles. It then applies all 256 pairs, changing `a` and `b` on a falling edge, and checks `p` twice. One cycle later `p` must still hold the previous product, and two cycles later it must equal `a * b`. Why two checks? The second alone would pass a design with 1-cycle latency. The first catches it.

```
RTL:          PASS: 256 vectors checked, 0 errors
sum = a ^ b:  FAIL: 256 vectors checked, 189 errors
```

For the error test we compiled the testbench against a copy of `rtl/full_adder.v` with `sum` changed to `a ^ b`. So 189 of the 512 checks failed.

1.2 Worked example

The last digit of Krish's roll number is 6, so A = 6 + 5 = 11 and B = 7.

```
          1 0 1 1   A = 11
        x 0 1 1 1   B = 7
---------------
          1 0 1 1   pp0 = A AND b0 (b0 = 1)
        1 0 1 1     pp1 = A AND b1 (b1 = 1), shifted 1
      1 0 1 1       pp2 = A AND b2 (b2 = 1), shifted 2
    0 0 0 0         pp3 = A AND b3 (b3 = 0), shifted 3
---------------
0 1 0 0 1 1 0 1   P = 77
```

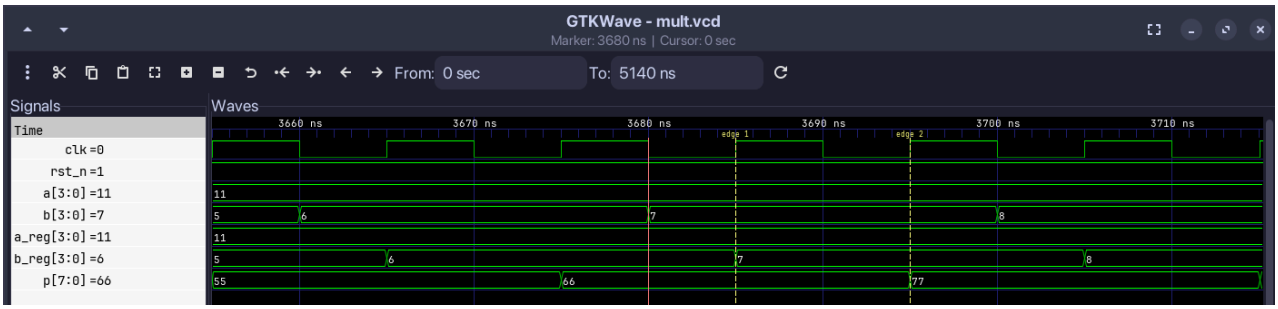


Figure 2: RTL simulation of 11 x 7. The two marked rising edges are the 2-cycle latency.

Through the array, row by row, where ri is the 4 bits row i passes down to row i+1:

Row	Adds	Sum (5 bits)	Product bit	Passed down
0	none	none	p[0] = pp0[0] = 1	r0 = 0101
1	pp1 1011 + r0 0101	1 0000	p[1] = 0	r1 = 1000
2	pp2 1011 + r1 1000	1 0011	p[2] = 1	r2 = 1001
3	pp3 0000 + r2 1001	0 1001	p[3] = 1	p[7:4] = 0100

This gives p = 0100 1101 = 77. In the RTL simulation `b` changes to 7 (with `a` = 11) at 3680 ns, `b_reg` takes it at the rising edge at 3685 ns, and `p` becomes 77 at the next rising edge at 3695 ns (Figure 2).

2 Logic synthesis

`synth/synth.tcl` runs the seven Yosys steps of the brief against the typical-corner library `sky130_fd_sc_hd__tt_025C_1v80.lib`.

Metric	Value
Total number of cells	65
Number of flip-flops	16 (`dfrtp_1`)
Total cell area (µm²)	751.971
Flip-flop area (µm²)	16 x 25.024 = 400.384
Flip-flop area (% of total)	53.24
3 most-used cell types	`dfrtp_1` 16, `nand2_1` 7, then a tie at 5 (`a21oi_1`, `nor2_1`, `xnor2_1`, `xor2_1`)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

The netlist simulation used the brief's command with `-DFUNCTIONAL -DUNIT_DELAY=#1` and the same testbench, so the 1-cycle and 2-cycle checks both ran on the gate-level netlist.

Figure 3 is the same 11 x 7 example on the netlist. Why does every edge sit later than in Figure 2? The SKY130 cell models use a 1 ns time unit and apply `UNIT_DELAY` to their flip-flop and latch primitives, so with `#1` every flip-flop output changes 1 ns after the clock edge. `b_reg` changes at 3686 ns and `p` becomes 77 at 3696 ns. The latency is still 2 cycles.

3 Static timing analysis

`sta/sta.tcl` runs the seven OpenSTA steps of the brief. It reads the clock period T from the `CLOCK_PERIOD` environment variable and uses 10 ns when it is not set.

At T = 10 ns the worst slack is 7.32 ns. So the minimum period is Tmin = 10 - 7.32 = 2.68 ns and the maximum clock frequency is Fmax = 1000 / Tmin MHz. At T = 2.68 ns the worst slack is 0.004 ns, and `sta/mult_array_timing.rpt` is from that run.

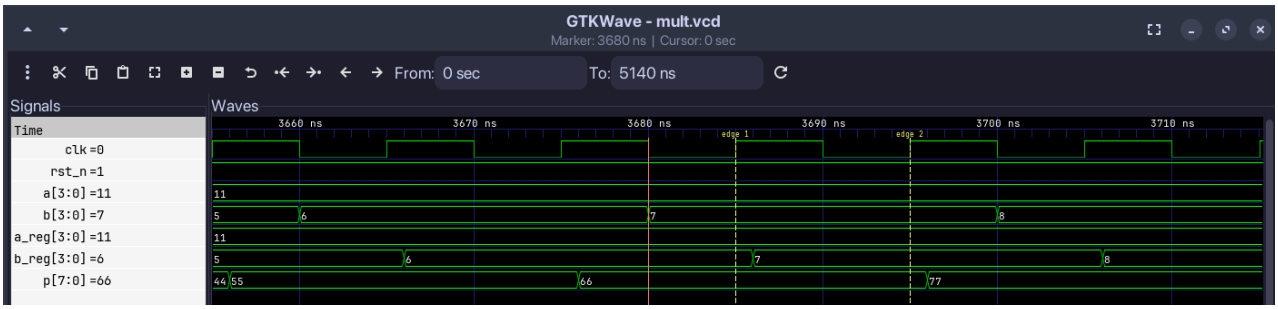


Figure 3: Netlist simulation of 11 x 7, with the same markers as Figure 2.

Metric	**Value**
Worst slack at T = 10 ns (ns)	7.32
Tmin (ns)	2.68
Fmax (MHz)	1000 / 2.68 = 373.1
Slack at T = Tmin (ns)	0.004
Critical path start point	`b_reg[0]` (cell `_102_`)
Critical path end point	`p[6]` (cell `_096_`, D = `gen_row3[3].fa.sum`)
Number of cells on the critical path	8 logic cells, 10 with both flip-flops
Latency (ns) = 2 x Tmin	5.36
Throughput (million results per second)	373.1

The critical path, from `sta/mult_array_timing.rpt`:

#	**Cell**	**Delay (ns)**	**Arrival (ns)**
1	`_102_ dfrtp_1` (clock to Q of `b_reg[0]`)	0.386	0.386
2	`_042_ and4_1`	0.251	0.638
3	`_048_ nor3_1`	0.217	0.855
4	`_050_ o21ai_0`	0.173	1.028
5	`_061_ maj3_1`	0.404	1.431
6	`_072_ xnor2_1`	0.166	1.598
7	`_082_ maj3_1`	0.389	1.987
8	`_087_ maj3_1`	0.380	2.367
9	`_089_ xnor3_1`	0.175	2.542
10	`_096_ dfrtp_1` (D of `p[6]`, setup 0.134)		

Which full adders does the path go through? `-flatten` removes the `full_adder` boundaries, so the cells do not map one to one onto the adders of the RTL. We simulated the netlist and the RTL together on all 256 inputs and matched each net on the path to the RTL signal it equals, or its inverse. Figure 4 shows the result. The path enters the array through $a_1 b_0$ and goes through 8 full-adder stages, carry, sum, carry, carry, sum, carry, carry and sum, to `p[6]`.

All 8 cells compute full-adder logic. The three `maj3` cells are full-adder carries and the `xnor3` cell is a full-adder sum. The `xnor2` cell finishes a sum that ABC split over two `xnor2` cells (`_071_` and `_072_`). The `and4` cell is the carry of row 1 adder 0, which is a half adder because its carry in is 0, so its carry is $a_0 b_1$ AND $a_1 b_0$. The `nor3` and `o21ai` cells together compute the sum of row 1 adder 1 and the carry of row 2 adder 0.

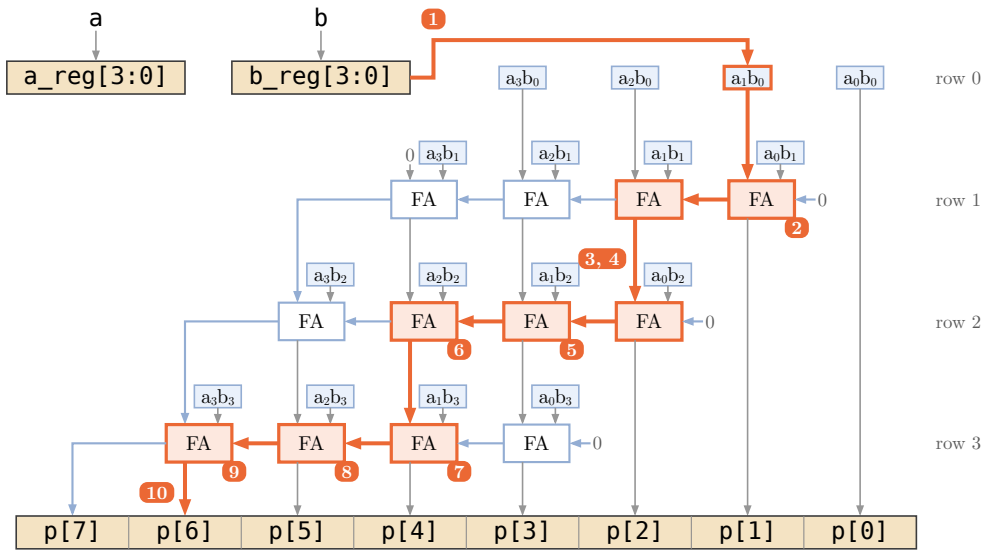


Figure 4: Part C critical path (orange) on the array of Figure 1. Badges number the cells as in the table above.

Now the estimate. Section 2 of the brief gives about 2N = 8 full-adder delays for N = 4, and the path has 8 full-adder stages in 8 cells. So it matches. The logic takes 2.542 - 0.386 = 2.156 ns after the clock-to-Q delay of `b_reg[0]`, or 0.27 ns per stage.

4 Physical design

LibreLane takes the RTL through synthesis, placement, clock tree synthesis, routing and sign-off. The brief sets CLOCK_PERIOD to 1.2 x Tmin = 1.2 x 2.68 = 3.216 ns, rounded up to 3.5 ns, and asks for 0 DRC errors, 0 LVS errors and a setup slack of at least 0. LibreLane checks setup at nine corners. They are typical (tt, 25 °C, 1.8 V), slow (ss, 100 °C, 1.6 V) and fast (ff, −40 °C, 1.95 V) transistors, each with minimum, nominal and maximum wire parasitics. We take the setup slack to be `timing__setup__ws`, the worst slack of the nine. The README lists every run.

4.1 Default parameters

With only the settings the brief lists, LibreLane uses its defaults. Its synthesis strategy is `AREA 0`, which optimises for area, and its resizer stops at 0.05 ns of estimated slack. At 3.5 ns the typical corner passes with 0.493 ns and the slow corner fails with −2.093 ns. So, following the brief, we raised CLOCK_PERIOD in 0.5 ns steps up to 6.5 ns. The slow corner never passed. From 5.5 ns on its slack stayed between −0.04 and −0.16 ns (grey line in Figure 5).

4.2 Slow-corner setup violation

The step that fixes setup is the resizer (Figure 6). After clock tree synthesis it upsizes gates and adds buffers on failing paths, timed with wire estimates since it runs before routing, and stops once the estimated slack reaches its margin. Figure 7 compares that estimate with the slack signed off after routing. Up to 5.5 ns the default flow's resizer could not reach 0. At 6.0 and 6.5 ns it reached +0.059 and +0.050 ns, and routing then took the slack to −0.043 and −0.155 ns. A longer period leaves the resizer less to fix, so it stops just above 0 and routing undoes the gain. The netlist is not the cause. LibreLane's synthesis gave the same 71 cells at every period.

4.3 Timing closure

So the fix has two parts (Table 1). The resizer margins leave room for routing, and a second resizer pass runs once global routing knows the wire lengths. `DELAY 0` shortens the paths the resizer starts from. At 3.5 ns it raises the resizer's estimate from −2.008 to −1.159 ns. Now, which of the two does the work? Table 2 tries each alone and both together at 5.0 ns.

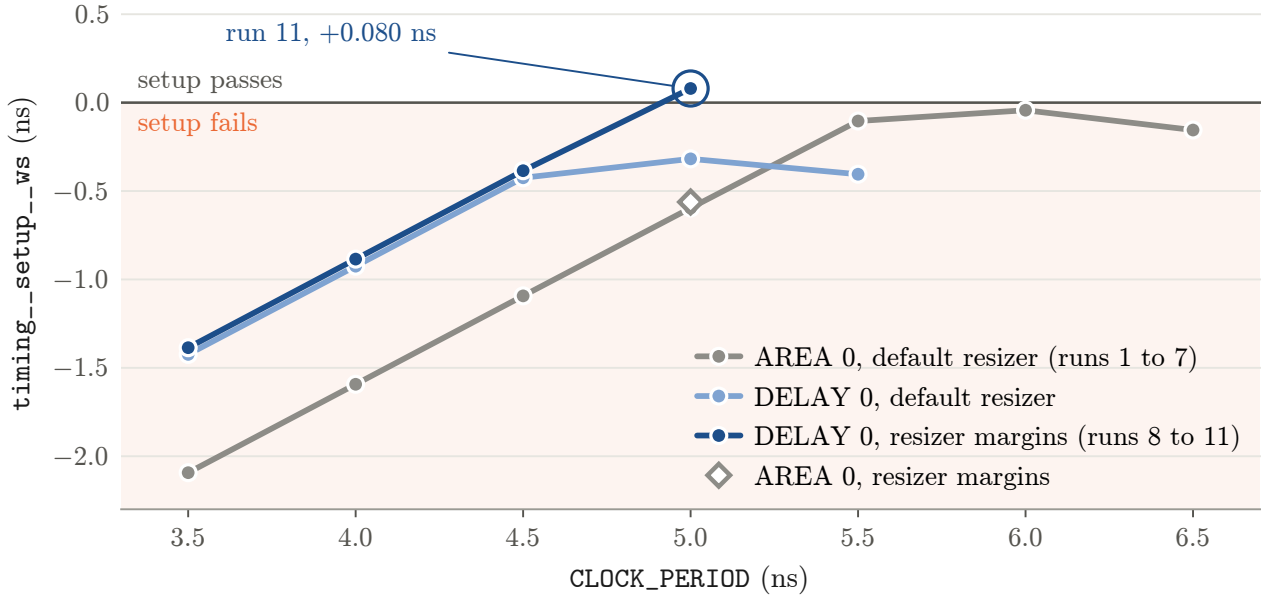


Figure 5: `timing__setup__ws` of every run against CLOCK_PERIOD. The ring marks run 11, the only run that passes.

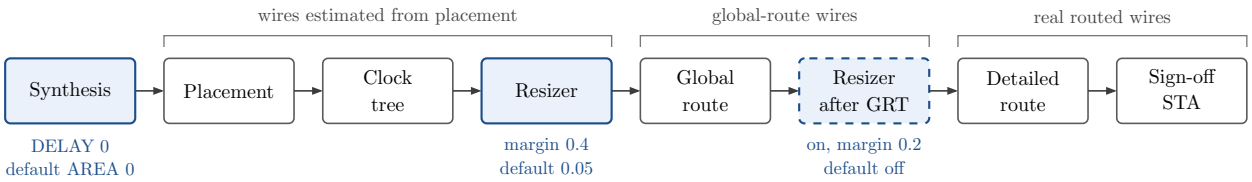


Figure 6: The LibreLane steps that decide the setup slack. Blue marks our settings, and the dashed pass is off by default. The first resizer times estimated wires, and sign-off times the routed ones.

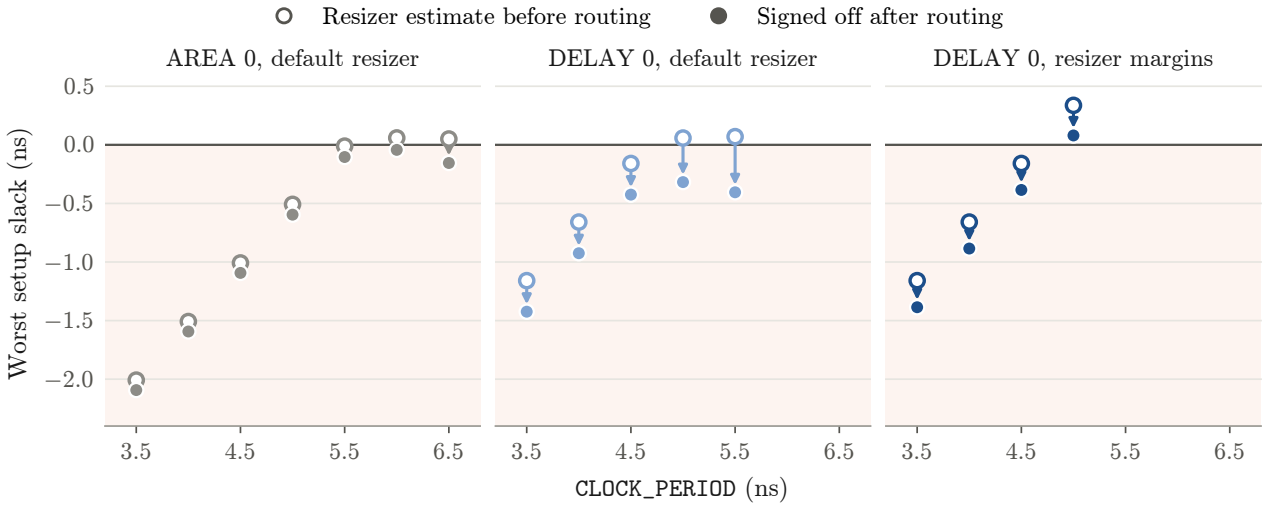


Figure 7: Worst setup slack of each run, one panel per flow, estimated by the resizer before routing (hollow) and signed off after routing (filled). Each arrow is what routing cost.

Table 1: The four settings we added to `config.json`. The first is the synthesis change and the other three are the resizer margins.

Setting	Default	Ours	Effect
`SYNTH_STRATEGY`	`AREA 0`	`DELAY 0`	Synthesis optimises for delay instead of area
`PL_RESIZER_SETUP_SLACK_MARGIN`	0.05	0.4	The resizer stops at 0.4 ns of estimated slack
`RUN_POST_GRT_RESIZER_TIMING`	false	true	A second resizer pass after global routing
`GRT_RESIZER_SETUP_SLACK_MARGIN`	0.025	0.2	The margin of that second pass

Table 2: Each change alone and both together, at CLOCK_PERIOD 5.0 ns.

Synthesis	Resizer	Estimate before routing (ns)	`timing__setup__ws` (ns)
`AREA 0`	default	−0.508	−0.596
`AREA 0`	margins	−0.508	−0.562
`DELAY 0`	default	+0.058	−0.318
`DELAY 0`	margins	+0.337	+0.080

Only both together pass. With `AREA 0` the resizer cannot reach 0 even at its default margin, so a larger margin has nothing to protect, and the post-routing pass gains only 0.034 ns. `DELAY 0` alone lets the resizer reach +0.058 ns, but its 0.05 ns margin leaves no room and routing takes the slack to −0.318 ns. With both, the resizer reaches +0.337 ns and +0.080 ns is left after routing. So we repeated the CLOCK_PERIOD steps from 3.5 ns with both changes. 3.5, 4.0 and 4.5 ns fail and 5.0 ns passes at all nine corners (dark blue line in Figure 5). 5.0 ns is the submitted design.

4.4 Final design

Table 3 gives the Part D metrics of the brief for the default flow at 3.5 ns and for the submitted design at 5.0 ns.

Table 3: Part D results of the default flow and the submitted design.

Metric	Default flow	Submitted
CLOCK_PERIOD used (ns)	3.5	5.0
Number of runs needed	tt passes in run 1, ss never passes (runs 1 to 7)	4 (runs 8 to 11)
Core area (µm²)	17759.5 (die 22500)	17759.5 (die 22500)
Cells after synthesis (Part B)	65	65
Cells after place and route	4791 (355 without fill and decap)	4692 (400 without fill and decap)
Kinds of cells added	decap, fill, tap, clock and timing buffers	decap, fill, tap, clock and timing buffers
Wirelength (µm)	2602	5089
Setup slack, `timing__setup__ws` (ns)	−2.093 (ss)	0.080 (ss)
Setup slack at tt (ns)	0.493	2.393
Post-route Fmax (MHz), from `setup__ws`	fails	1000 / (5.0 - 0.080) = 203.3
Post-route Fmax at tt (MHz)	1000 / (3.5 - 0.493) = 332.5	1000 / (5.0 - 2.393) = 383.6
Fmax from Part C (MHz)	373.1	373.1
DRC errors	0 (Magic and KLayout)	0 (Magic and KLayout)
LVS errors	0	0

The submitted design passes at every corner (Figure 8). The price is size (Table 4).

Table 4: What the submitted design costs against the default flow.

Metric	Default flow	Submitted	Ratio
Cells after LibreLane's synthesis	71	117	1.65
Logic area after synthesis (µm²)	948.4	1402.6	1.48
Routed wirelength, `route__wirelength` (µm)	2602	5089	1.96
Fmax at tt (MHz)	332.5	383.6	1.15

Where do the 4692 instances come from? Table 5 sorts every instance by kind. Only the last row is the multiplier itself, and the rest is what the flow added.

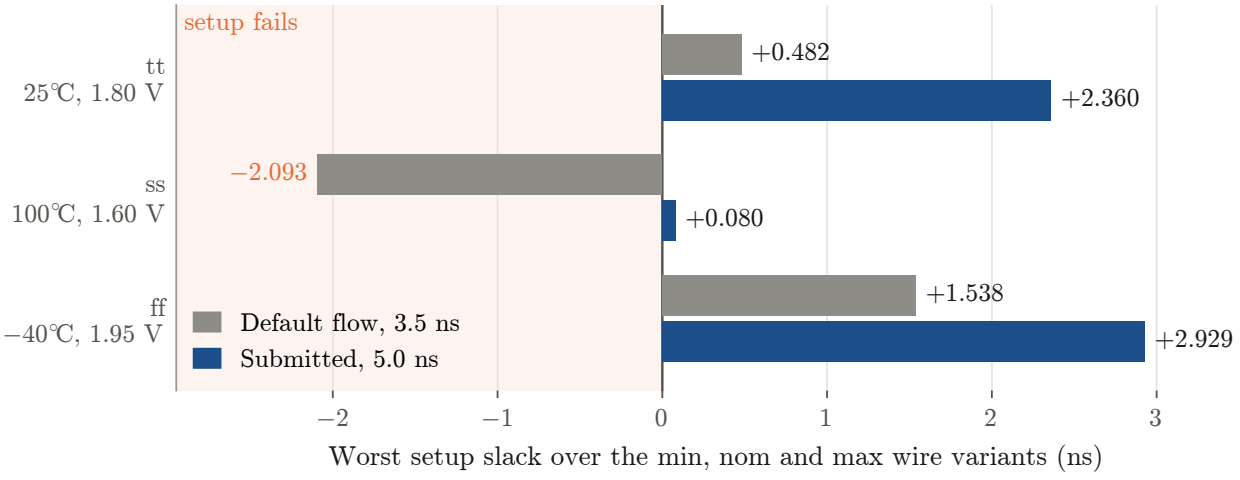


Figure 8: Worst setup slack at each transistor corner, taking the worst of its min, nom and max wire variants.

Table 5: Every instance of the two layouts, from the final netlists.

Kind	What it does	Default flow	Submitted
`decap_3`	Capacitor between VPWR and VGND that supplies charge locally when cells switch	4157	4009
`fill_1`, `fill_2`	Closes the gaps in each row so the wells and power rails are continuous	279	283
`tapvpwrvgnd_1`	Ties the n-well to VPWR and the substrate to VGND, against latch-up	245	245
`clkbuf_16`	Clock tree, one root and two leaves	3	3
Input and output buffers	Isolate the ports	17	17
Fanout buffers	Drive heavily loaded nets	3	6
Setup repair buffers	Speed up failing paths	15	12
Hold delay cell	Slows down a path that is too fast	1	0
Logic cells and flip-flops	The multiplier	71	117
Total		4791	4692

At the slow corner LibreLane still reports 22 max-slew warnings, all on the `rst_n` reset net, whose 0.784 ns transition at the flip-flop RESET_B pins exceeds the 0.75 ns limit. They do not touch setup, hold, DRC or LVS.

4.5 Layout

Figure 9 draws the submitted layout from the final DEF, with every cell at its real position and size and every signal wire on its real route. Only 155 of the 4692 cells belong to the design itself, the logic cells, flip-flops and buffers, and they sit in a band through the middle of the core. The rest of each row holds decap, fill and tap cells, drawn as the grey background. The orange nets are the critical path that LibreLane's sign-off timing reports, from `a_reg[2]` to `p[7]` through 7 logic cells.

Each row of cells sits between two met1 power rails, alternating VPWR and VGND, so neighbouring rows share a rail (Figure 10). Signal wires run on top of the cells, met1 horizontally and met2 vertically, and vias join the two layers.

The `clk` pin feeds a root `clkbuf_16` near the centre, which drives two leaf buffers of 8 flip-flops each, so all 16 flip-flops are two buffers from the pin (Figure 11). At the typical corner the worst pair in the skew report sees the clock 0.358 and 0.326 ns after the edge at the pin, a skew of 0.032 ns.

Figure 12 shows the two KLayout screenshots the brief asks for, with every layer of the GDS. The 4009 decap and 283 fill cells cover 87% of the core area and would hide the logic cells and wires, so they are left out of the view. The tap cells stay.

5 Questions

5.1 Logic cells after synthesis

Synthesis produced 65 - 16 = 49 logic cells for 16 AND gates and 12 full adders. Why not 28? ABC maps logic functions, not the RTL instances. With `-flatten` there is no `full_adder` boundary left, so ABC splits full adders into several smaller gates and merges most AND gates into complex cells. There are only 2 plain `and2` cells, and the `and4`, `a21oi`, `a22oi` and `a211oi` cells combine ANDs with adder logic. Only 6 cells compute a whole full-adder output, the 4 `maj3` carry cells and the 2 `xnor3` sum cells. Constant inputs also remove logic. The first carry of each row (`c1[0]`, `c2[0]`, `c3[0]`) is 0, and so is the top bit `r0[3]` that row 0 passes down, so four adders are really half adders. The 2 `lpflow_isobufsrc_1` cells compute `A & !SLEEP`, which ABC used as an AND with one inverted input.

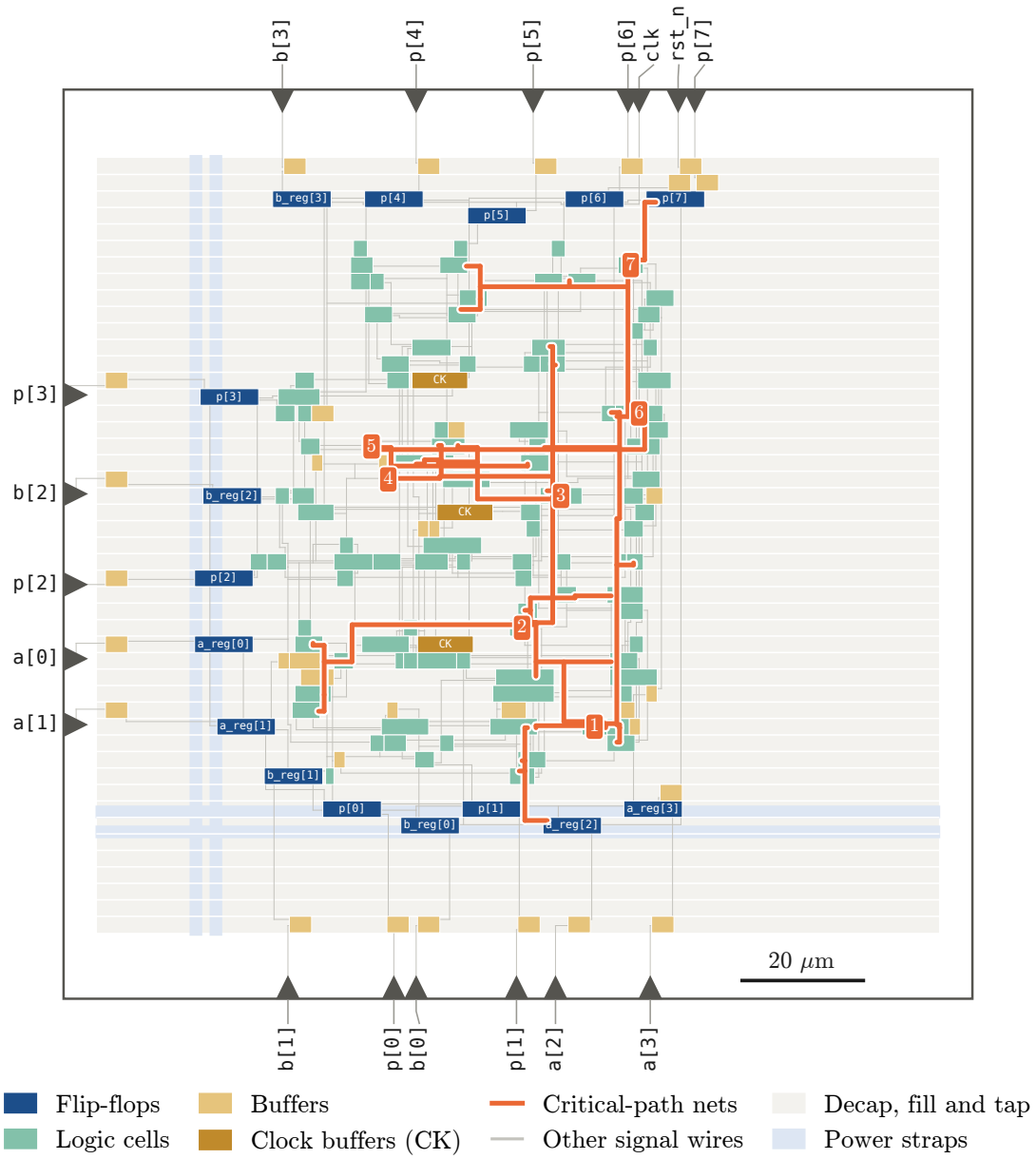


Figure 9: Submitted layout, 150 x 150 µm. The critical path is orange, and its badges 1 to 7 number the cells in signal order.

5.2 Critical path

It runs from the `b_reg[0]` register to the `p[6]` register, whose input is the sum of the last full adder of row 3, through 8 full-adder stages in 8 logic cells. That matches the estimate of about 2N = 8 full-adder delays (Figure 4).

5.3 Flip-flop area

Flip-flops are 400.384 of 751.971 µm², which is 53.24%. Why so much? One `dfrtp_1` is 25.024 µm², the area of about 6.7 `nand2_1` cells (3.754 µm² each), because a flip-flop with asynchronous reset holds two latches and the reset logic, and the design has only 49 small logic cells for 16 flip-flops. Logic grows as N² (N² AND gates and N(N-1) full adders) while the registers grow as 4N, so the flip-flop share falls as N grows.

5.4 Post-route Fmax

For the submitted design, post-route Fmax from `timing__setup__ws` is 1000 / (5.0 - 0.080) = 203.3 MHz, against 373.1 MHz from Part C. Where does the difference come from? Most of it is the corner. `timing__setup__ws` comes from the slow corner (ss, 100 °C, 1.6 V), while Part C used only the typical

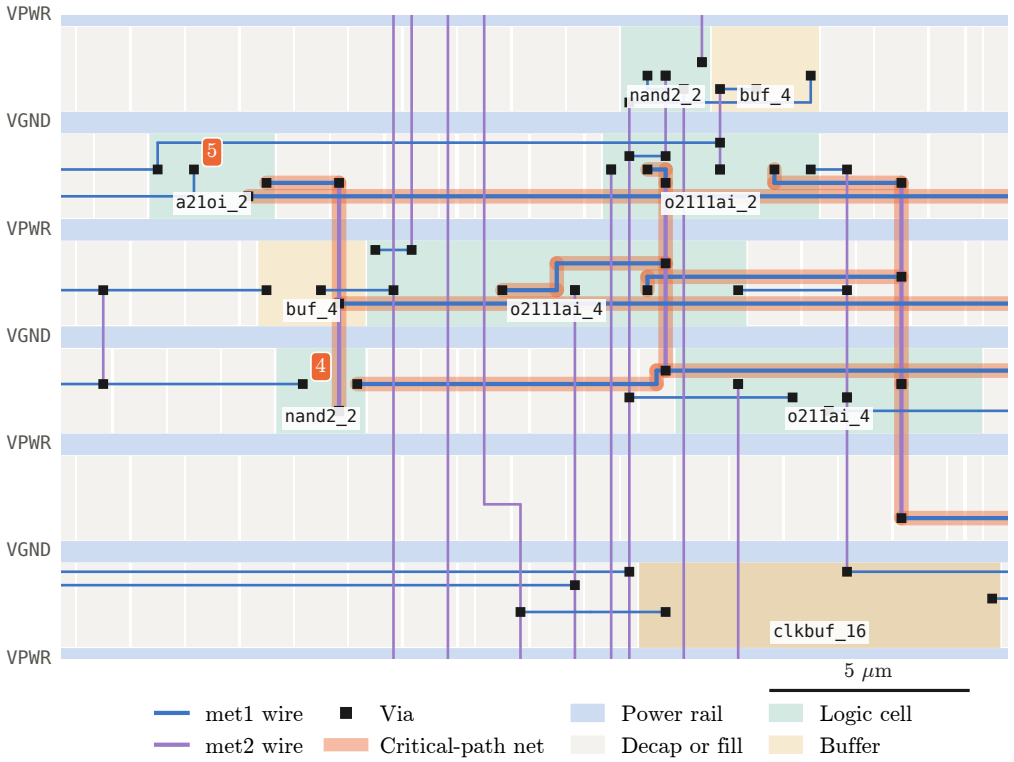


Figure 10: Six cell rows, 24 µm wide, around critical-path cells 4 and 5. Each cell shows its type.

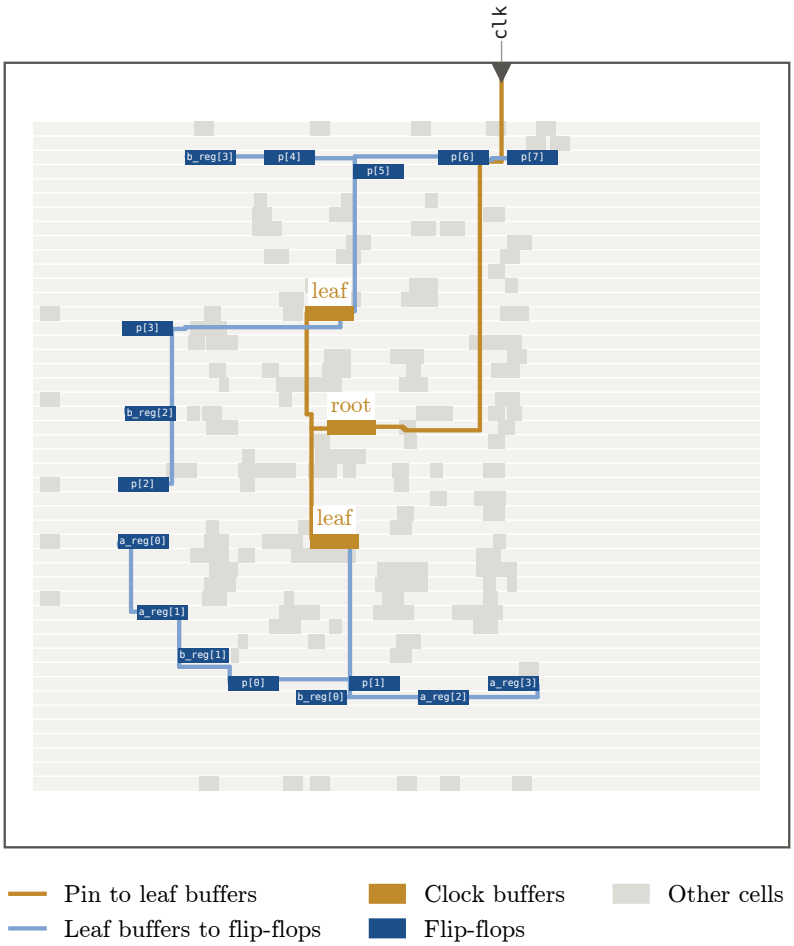


Figure 11: Clock tree of the submitted design, drawn from the routed clock nets in the DEF.

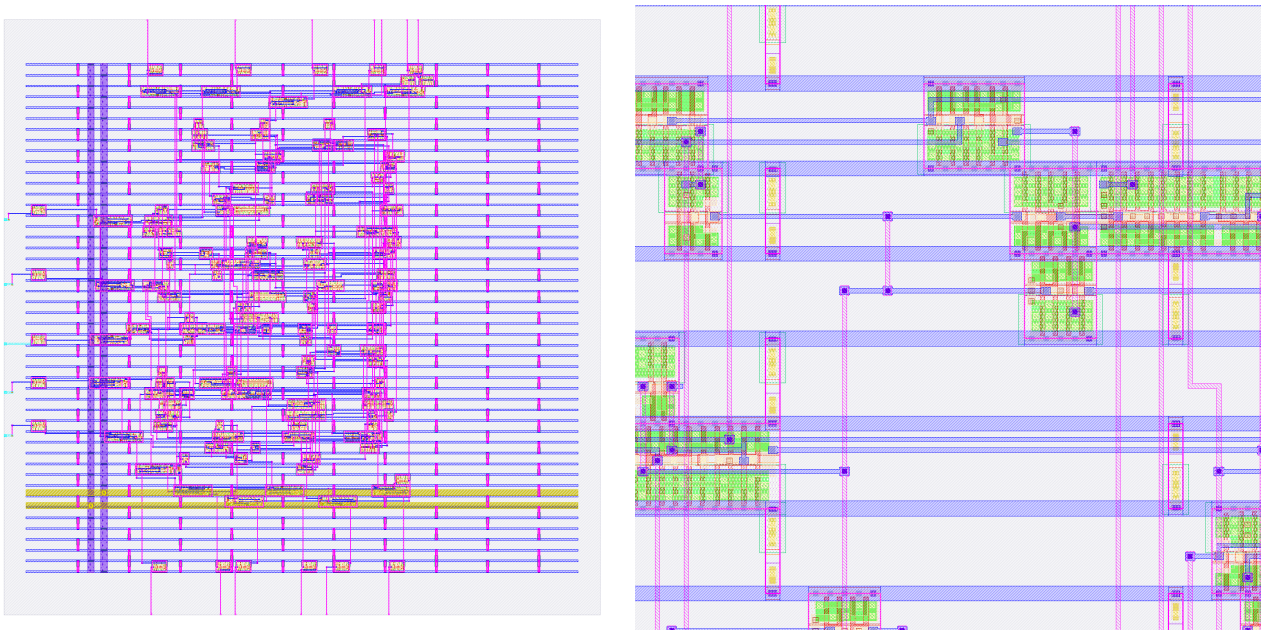


Figure 12: KLayout screenshots of the final GDS with all layers, without decap and fill cells. Left, the whole die. Right, a 20 µm window over the logic.

corner. The critical path of Figure 9 is the same at both corners, but its data arrives at 5.013 ns at ss against 2.594 ns at tt. At a lower supply and higher temperature each transistor gives less current, so every gate is about twice as slow (Figure 13). The rest is the 0.25 ns clock uncertainty that LibreLane's constraints add and Part C did not, real wire resistance and capacitance, and clock skew from the clock tree.

Coming back to the typical corner, the submitted layout reaches 1000 / (5.0 - 2.393) = 383.6 MHz, slightly higher than Part C. LibreLane's delay-oriented synthesis built a critical path of 7 logic cells against 8 in Part B, and the resizer used stronger `_2` and `_4` cells, which together gain more than the clock uncertainty costs. The default flow uses area-oriented synthesis, and at tt it reaches 332.5 MHz, lower than Part C as expected. Its critical path runs from `a_reg[3]` to `p[7]` through 9 logic cells and a fanout buffer. Its effective period is 0.331 ns longer than in Part C, and Table 6 splits that difference.

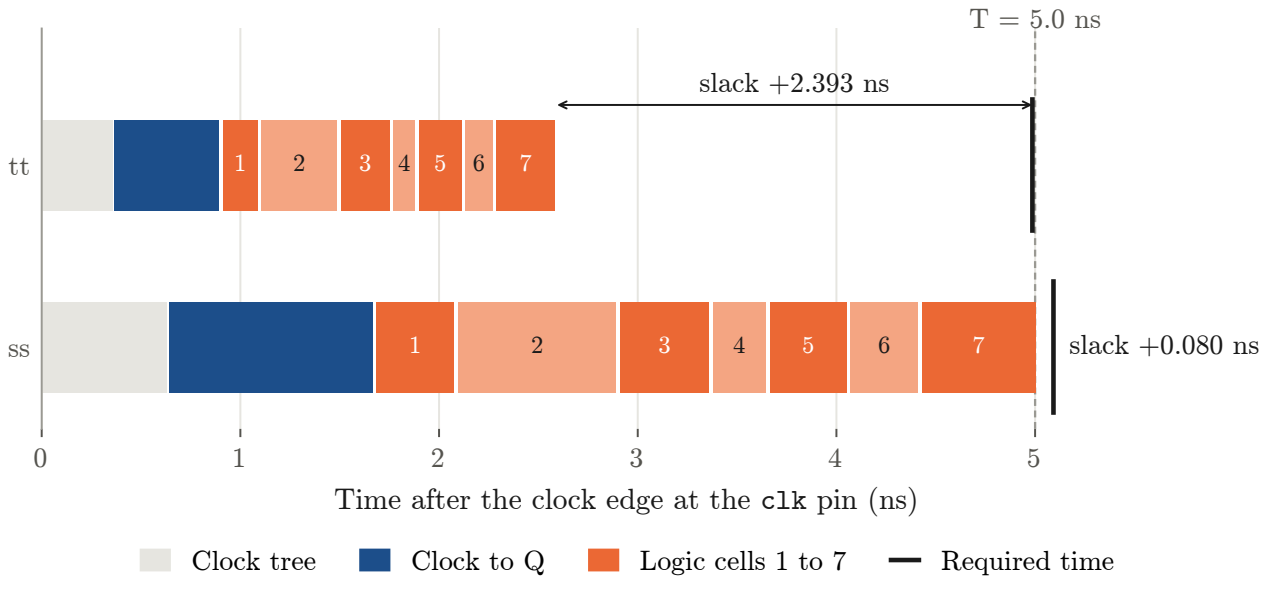


Figure 13: The critical path at `nom_tt_025C_1v80` and `max_ss_100C_1v60`, the corner of `timing__setup__ws`. Cells 1 to 7 are numbered as in Figure 9.

Table 6: From the Part C period to the default flow's effective period at tt.

Term	**ns**
Part C, arrival plus setup (2.542 + 0.134)	2.676
Clock uncertainty that LibreLane's constraints add	+0.250
Data path, a different netlist with real wires and one more buffer	+0.090
Clock skew from the clock tree	+0.015
Smaller setup time of the larger `dfrtp_2` flip-flop	−0.024
Default flow effective period (3.5 - 0.493)	3.007

5.5 DRC and LVS

DRC checks the layout geometry against the foundry's rules, such as minimum widths, spacings, enclosures and densities on each layer. A violation means the foundry may not be able to print the shapes reliably, which can cause shorts or opens. LVS extracts the transistors and connections from the layout and compares them with the netlist. A mismatch means the layout is a different circuit from the one we simulated and timed. Both must pass. DRC says the chip can be made and LVS says it is the right chip. Our design has 0 DRC errors in Magic and KLayout and 0 LVS errors in Netgen.

5.6 Scaling to 8 bits

The array grows as N^2 while the critical path grows as 2N (Table 7). From Part C, the 4-bit logic delay is 2.542 - 0.386 = 2.156 ns. Taking that as the 8 full-adder delays of the estimate, even though ABC merged them into 8 cells of mixed kinds, gives 2.156 / 8 = 0.2695 ns each. So an 8-bit array needs 16 x 0.2695 + 0.386 + 0.134 = 4.83 ns per cycle.

Table 7: The 4-bit design against an 8-bit array with the same cells.

	N = 4	**N = 8**
AND gates, N^2	16	64
Full adders, N(N - 1)	12	56
Full-adder delays on the critical path, about 2N	8	16
Flip-flops, 4N	16	32
Clock period (ns)	2.68 (Part C)	4.83 (estimate)
Fmax (MHz)	373.1	207

Acknowledgements

We sat together and did everything, with the help of Claude and GPT at every step.